



TECNOLÓGICO
NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE CELAYA
Departamento de Ingeniería en Sistemas y Computación



TAREA 1 – COMPETENCIA 2
TAREA - TODO LIST (FIREBASE)

ASIGNATURA:

TÓPICOS AVANZADOS DE PROGRAMACIÓN

GRUPO:

A

ESPECIALIDAD:

ING. SISTEMAS COMPUTACIONALES

DOCENTE:

JOSÉ JESUS SANCHEZ FARÍAS

ALUMNO(A):

ALAN DANIEL MENDEZ JIMENEZ

ING. SISTEMAS
COMPUTACIONALES



TECNOLÓGICO
NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE CELAYA
Departamento de Ingeniería en Sistemas y Computación



Contenido

Introducción	3
Desarrollo e Implementación Técnica.....	4
Investigación, Instalación y Configuración de Google Cloud Firestore	4
Proceso de Integración:	5
Diseño de la Estructura de Datos (NoSQL)	6
Lógica de Negocio y Manejo de Concurrencia	7
Programación Asíncrona:	7
Seguridad y Segregación de Datos (Multi-tenancy)	7
Funcionalidades de la Interfaz de Usuario Avanzada	8
Dashboard Estadístico:.....	8
Exportación e Interoperabilidad (PDF y Excel).....	9
Conclusión	10
Referencias Bibliográficas	11

ING. SISTEMAS
COMPUTACIONALES

Introducción

El seguimiento de actividades, la gestión del tiempo y la productividad son métricas vitales para cualquier organización moderna.



"To-Do List Enterprise", un Sistema de Gestión de Tareas.

El propósito de este desarrollo es demostrar la capacidad de diseñar una arquitectura de software que integre una Interfaz Gráfica de Usuario (GUI) rica y responsiva construida con el framework JavaFX, operando en sincronía con Google Cloud Firestore, una base de datos NoSQL alojada en la infraestructura de nube de Google. Esta convergencia tecnológica permite el almacenamiento seguro, la escalabilidad horizontal y la persistencia de datos descentralizada.

A lo largo de este proyecto, se abordaron problemáticas críticas del desarrollo de software a nivel empresarial, la interoperabilidad mediante la exportación de reportes estructurados a formatos estandarizados (PDF y Microsoft Excel).

El resultado es una aplicación completa que trasciende el concepto básico de una lista de tareas para convertirse en una herramienta integral de gestión y reporte.



ING. SISTEMAS
COMPUTACIONALES



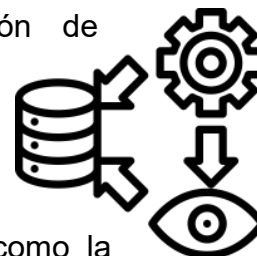
TECNOLÓGICO
NACIONAL DE MÉXICO

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE CELAYA
Departamento de Ingeniería en Sistemas y Computación



Desarrollo e Implementación Técnica

El ciclo de desarrollo se fundamentó en la separación de responsabilidades mediante el patrón arquitectónico MVC (Modelo-Vista-Controlador), apoyado por el patrón DAO (Data Access Object) para la capa de persistencia y el patrón Singleton para la gestión centralizada de recursos críticos como la sesión del usuario y el canal de conexión a la base de datos.



Investigación, Instalación y Configuración de Google Cloud Firestore



Para cumplir con el requerimiento de almacenamiento en la nube, se investigó y seleccionó Google Cloud Firestore debido a su naturaleza orientada a documentos, la cual ofrece una flexibilidad superior frente a las bases de datos relacionales tradicionales (SQL) al modelar entidades complejas como listas de categorías dentro de una misma tarea.

ING. SISTEMAS
COMPUTACIONALES



Proceso de Integración:

1. Aprovisionamiento en Firebase Console:

Se creó un nuevo proyecto en la plataforma de Google Firebase. Se inicializó el servicio "Firestore Database" en modo de producción, seleccionando un clúster geográfico cercano

2. Generación de Credenciales de Seguridad (Service Account):

Para permitir que la aplicación Java de escritorio se comuniquen de forma segura con los servidores de Google sin exponer la base de datos al público, se generó una cuenta de servicio. Desde la consola de Google Cloud, se emitió una clave privada en formato JSON (`serviceAccountKey.json`). Este archivo contiene identificadores criptográficos exclusivos del proyecto.

Fragmento de configuración de SDK de Admin

☐ Node.js ☒ Java ☐ Python ☐ Go

```
FileInputStream serviceAccount =  
    new FileInputStream("path/to/serviceAccountKey.json");  
  
FirebaseOptions options = new FirebaseOptions.Builder()  
    .setCredentials(GoogleCredentials.fromStream(serviceAccount))  
    .build();  
  
FirebaseApp.initializeApp(options);
```

3. Configuración del Entorno Java:

Se añadió la dependencia oficial `firebase-admin` al gestor de paquetes del proyecto (Gradle/Maven).

```
testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:${junitVersion}")  
  
implementation("com.google.firebase:firebase-admin:9.2.0")  
implementation("org.slf4j:slf4j-simple:2.0.7")  
  
implementation("com.itextpdf:kernel:7.2.5")
```



4. Implementación del Singleton de Conexión (`DBConnection.java`):

La inicialización del SDK de Firebase se programó para ejecutarse una única vez al arrancar la aplicación. Se utilizó la clase `GoogleCredentials.fromStream()` para leer el archivo JSON, inyectando estas credenciales en un objeto `FirebaseOptions`. Posteriormente, se llamó a `FirebaseApp.initializeApp()`, obteniendo finalmente la instancia global de `Firestore` que servirá como canal de comunicación de toda la aplicación.

Diseño de la Estructura de Datos (NoSQL)

La información se estructuró jerárquicamente en las siguientes colecciones principales:

Colección `users`:

Almacena la identidad y autenticación. Contiene documentos con los campos `username` y `password`.

Colección `categories`:

Funciona como un catálogo dinámico y personalizado. Contiene `id`, `name` y el `owner` (dueño de la categoría), permitiendo que cada usuario tenga sus propias etiquetas sin interferir con las de otros.

Colección `tasks`:

Cada documento almacena metadatos: `id` autogenerado, `name`, `description`, `status` (Pendiente, En Progreso, Completada), `dueDate` (Fecha de vencimiento), `creationDate`, `owner` y un arreglo nativo (Array) denominado `categories` que almacena los identificadores o nombres de las etiquetas asociadas.



Lógica de Negocio y Manejo de Concurrency

Dado que las operaciones de red hacia Firebase poseen una latencia inherente, ejecutar estas llamadas en el hilo principal de JavaFX (JavaFX Application Thread) provocaría que la interfaz gráfica se congelara. Para mitigar esto, se implementó la capa `TaskService`.

Programación Asíncrona:

Todas las transacciones CRUD (Crear, Leer, Actualizar, Borrar) fueron encapsuladas dentro de objetos `javafx.concurrent.Task`.

Al ejecutarse mediante `new Thread(worker).start()`, la carga de trabajo se deriva a un hilo en segundo plano. Los controladores (Controllers) utilizan los métodos de retrollamada (callbacks) como `setOnSucceeded()` para actualizar la interfaz visual únicamente cuando Firestore confirma que la operación ha concluido exitosamente.

Seguridad y Segregación de Datos (Multi-tenancy)

El sistema gestiona múltiples usuarios compartiendo la misma base de datos física, pero manteniendo un aislamiento lógico estricto. La clase `SessionManager` guarda en memoria la identidad del usuario autenticado.

En cada consulta generada por el `TaskDAOImpl`, se inyecta programáticamente una cláusula de filtrado restrictiva: `.whereEqualTo("owner", currentUser)`. Esto garantiza a nivel de API que el flujo de datos descargado pertenece exclusiva y privadamente al usuario en sesión.





Funcionalidades de la Interfaz de Usuario Avanzada

Ventanas Modales y CRUD Completo:

La interacción principal se extrajo de la pantalla base hacia ventanas modales de diálogo (``Modality.APPLICATION_MODAL``). Esto concentra la atención del usuario en formularios validados que previenen la entrada de datos inconsistentes (por ejemplo, bloqueando dinámicamente fechas anteriores al día actual en el componente ``DatePicker``).

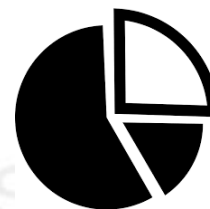
Motor de Filtrado en Memoria:

Se implementó una instancia de ``FilteredList`` en el ``TaskController``.

Este motor evalúa un predicado (Predicate) cada vez que el usuario teclea en la barra de búsqueda o cambia el selector de categorías. Al procesar las condiciones lógicas en la memoria RAM del equipo cliente (combinando ``OR`` para texto y ``AND`` para categorías), se logra una búsqueda instantánea sin consumir cuota de lectura en la nube de Firebase.

Dashboard Estadístico:

Se desarrolló un módulo de Inteligencia de Negocios (BI) básico utilizando ``PieChart`` y ``BarChart``. El sistema recupera el conjunto de datos, evalúa el atributo ``creationDate`` mediante el API ``java.time`` de Java 8+, y agrupa dinámicamente las tareas para visualizar la productividad (Completadas vs No Completadas) y la carga de trabajo segmentada por categoría, permitiendo filtrar la línea temporal (semana, mes, año).





Exportación e Interoperabilidad (PDF y Excel)

La capacidad de extraer información del sistema se delegó a la clase especializada `ReportService`, la cual integra dos potentes librerías de terceros:

1.iText (v5):



Empleada para renderizar de manera programática un archivo de formato de documento portátil (PDF). Se construyen objetos `Document`, tipografías estandarizadas con `FontFactory` y tablas dinámicas mediante `PdfPTable`, garantizando una presentación formal e inalterable orientada a la impresión o auditoría.

2.Apache POI:



Utilizada para la generación nativa de hojas de cálculo de formato Office Open XML (.xlsx). A través de las interfaces `Workbook`, `Sheet`, `Row` y `Cell`, los datos en memoria se iteran y transcriben en una matriz bidimensional, aplicando algoritmos de auto-ajuste de columnas (`autoSizeColumn`) para facilitar el análisis posterior de los datos por parte de los administradores.

ING. SISTEMAS
COMPUTACIONALES



Conclusión

La culminación del proyecto "To-Do List Enterprise" donde se logró trascender la programación local convencional mediante la correcta orquestación de servicios en la nube con Google Cloud Firestore.

La investigación realizada sobre los procesos de conexión de Firebase fue buena para comprender que la persistencia moderna exige protocolos de autenticación asimétrica (Service Accounts) y diseños de esquemas flexibles basados en documentos.

Desde la perspectiva de la ingeniería de software, la aplicación de patrones de diseño garantizó un código modular, altamente cohesivo y de bajo acoplamiento.

El control de los hilos mediante ``javafx.concurrent.Task`` aseguró una experiencia de usuario óptima (UX), evitando el bloqueo de la vista gráfica durante las latencias de red. Asimismo, la integración de librerías como Apache POI e iText demostró la capacidad de acoplar ecosistemas de código abierto para resolver necesidades corporativas reales de reporte e interoperabilidad.

En resumen, el software construido no es únicamente un registro digital de actividades, sino una arquitectura empresarial completa, escalable y segura, que sienta bases sólidas para el desarrollo de futuras soluciones tecnológicas multiplataforma de gran envergadura.

Referencias Bibliográficas

Google Cloud Platform. (2024). Cloud Firestore Documentation: Understand Cloud Firestore and add it to your server*. Alphabet Inc. Recuperado de: [https://firebase.google.com/docs/firestore]

(<https://firebase.google.com/docs/firestore>)

Oracle Corporation. (2023). Java Platform, Standard Edition JavaFX API Specification*. Recuperado de: [https://openjfx.io/javadoc/17/]

(<https://openjfx.io/javadoc/17/>)

iText Group NV. (2022). iText 5 Core API Documentation for Java Developers. Recuperado de: [https://itextpdf.com/en/resources/api-documentation]

(<https://www.google.com/search?q=https://itextpdf.com/en/resources/api-documentation>)

The Apache Software Foundation. (2024). Apache POI - the Java API for Microsoft Documents Component Reference. Recuperado de: [https://poi.apache.org/]

(<https://poi.apache.org/>)

Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). Java Concurrency in Practice*. Addison-Wesley Professional.

Schildt, H. (2018). Java: The Complete Reference, Eleventh Edition*. McGraw-Hill Education.